Roteiro de fala — Defesa do TCC

Autosim: DSL para o Ensino de Autômatos Finitos

Tempo planejado: **13:30** · Limite: **15 minutos** · 42 slides

Como usar: a coluna *Tempo* traz o tempo do slide e, abaixo, o tempo acumulado ao final dele. Use o acumulado para conferir o ritmo durante a fala. Os tempos já incluem as trocas de slide e as pausas, e sobra 1:30 para imprevistos. Nos slides de *Roteiro* do início de cada seção, diga só a frase de transição e avance.

Slide	Tempo	Fala
Abertura		**0:27**
1 — Capa	**0:15** até 0:15	Boa noite a todos. Somos João Thomaz Vieira e Klaus Siegfried Beckmann e vamos apresentar a Autosim, uma ferramenta baseada em DSL para o ensino de autômatos finitos, orientada pelo professor Diógenes Furlan.
2 — Roteiro	**0:12** até 0:27	A apresentação segue a estrutura da monografia: introdução, fundamentação, trabalhos relacionados, projeto da linguagem, implementação, resultados e conclusão.
1. Introdução		**2:09**
3 — Roteiro	**0:03** até 0:30	Começando pela introdução.
4 — Contexto e motivação	**0:40** até 1:10	Autômatos finitos e linguagens formais são um dos pilares da Ciência da Computação, mas a disciplina é conhecida por ser difícil. Morazán e Antunez apontam o porquê: na programação, o aluno escreve, executa e vê o resultado na hora; em autômatos, ele projeta e prova a correção no papel, sem poder experimentar nem receber retorno imediato. Isso afasta o formalismo da prática.
5 — O problema	**0:32** até 1:42	Existem ferramentas de apoio, e a mais conhecida é o JFLAP, mas elas são exclusivamente visuais: o autômato é montado arrastando e soltando estados. Assim, a descrição não é texto, o que dificulta versionar e compartilhar, não há validação formal, e o aluno que erra não recebe um diagnóstico preciso. Falta unir o rigor do texto, o diagnóstico de um compilador e a interatividade da simulação.
6 — A proposta: Autosim	**0:32** até 2:14	Nossa proposta é a Autosim, que acopla três partes: uma DSL textual, declarativa e em português, para AFDs e AFNs; um compilador pedagógico, que aponta os erros com linha e coluna; e uma interface gráfica que desenha o diagrama e anima a simulação passo a passo. A contribuição central é esse acoplamento: o aluno escreve o texto e a interface é gerada automaticamente.
7 — Objetivos	**0:22** até 2:36	O objetivo geral é integrar descrição textual, compilação pedagógica e simulação visual em um único ambiente. Para isso, projetamos a DSL, implementamos o compilador, desenvolvemos o simulador, construímos a interface e validamos tudo com autômatos clássicos e cenários de erro.
2. Autômatos Finitos e Linguagens Formais		**1:25**
8 — Roteiro	**0:03** até 2:39	Agora, a fundamentação, começando pelos autômatos.

Slide	Tempo	Fala
9 — Autômatos finitos	**0:32** até 3:11	Um AFD é uma quíntupla: estados, alfabeto, função de transição, estado inicial e estados de aceitação. Ele reconhece as cadeias que, partindo do estado inicial, terminam em um estado final. No AFN, a transição leva a um conjunto de estados e pode haver transições vazias, as transições épsilon. Os dois têm o mesmo poder: reconhecem exatamente as linguagens regulares.
10 — Exemplo prático	**0:30** até 3:41	No diagrama, os círculos são os estados, a seta marca o inicial, o círculo duplo é o final, e as arestas são a função de transição, também mostrada na tabela. Esse autômato aceita as cadeias que contêm `ab`: aceita `ab` e `aab`, rejeita `ba` e a cadeia vazia. Esse exemplo volta na DSL e na simulação.
11 — Hierarquia de Chomsky	**0:20** até 4:01	Na Hierarquia de Chomsky, que classifica as linguagens pela forma das regras, o tipo 3 reúne as gramáticas regulares, que geram exatamente as linguagens reconhecidas pelos autômatos finitos. É nesse nível que o trabalho atua.
3. Compiladores e Linguagens de Domínio Específico	**0:55**	
12 — Roteiro	**0:03** até 4:04	Em seguida, compiladores e DSLs.
13 — Fases de análise	**0:30** até 4:34	A Autosim tem um compilador, então vale lembrar as fases de análise. A léxica agrupa os caracteres em *tokens*. A sintática organiza os *tokens* em uma árvore sintática abstrata, a AST. E a semântica verifica o que a gramática não captura, como referências a algo não declarado.
14 — DSL	**0:22** até 4:56	Uma DSL, segundo Fowler, é uma linguagem de expressividade limitada, voltada a um domínio específico. Ela pode ser interna, embutida em outra linguagem, ou externa, com sintaxe própria e compilador dedicado, que é o caso da Autosim.
4. Rust como Linguagem de Implementação	**0:25**	
15 — Roteiro	**0:03** até 4:59	Sobre a linguagem de implementação:
16 — Por que Rust?	**0:22** até 5:21	Escolhemos Rust pelo desempenho comparável a C e C++, com segurança de memória garantida na compilação e sem coletor de lixo. O sistema de tipos ajuda a representar *tokens* e nós da AST, e o ecossistema já oferece bibliotecas maduras para *lexing*, *parsing* e interface gráfica.
5. Trabalhos Relacionados	**0:58**	
17 — Roteiro	**0:03** até 5:24	Agora, o que já existe.
18 — Panorama	**0:30** até 5:54	Analisamos cinco trabalhos. O JFLAP é a referência mundial, mas é só gráfico. O GAM é limitado a 10 estados e 3 símbolos. O simulador da UFSC roda na web, mas a construção segue gráfica. O AutomatoGraph gera a gramática regular, mas a simulação é parcialmente visual. E o trabalho de Bruzzone, em Rust, não é voltado ao ensino.

Slide	Tempo	Fala
19 — Análise comparativa	**0:25** até 6:19	Lado a lado: nenhuma das ferramentas de ensino que analisamos tem descrição textual, versionamento ou compilador com diagnóstico, e a única com versionamento não é voltada ao ensino. A Autosim reúne as cinco características. É essa lacuna que o trabalho ocupa.
6. Projeto da DSL		**1:18**
20 — Roteiro	**0:03** até 6:22	Passando para o projeto da linguagem.
21 — A linguagem: um exemplo	**0:34** até 6:56	Este é o mesmo autômato do exemplo anterior, escrito na Autosim. O arquivo `.asl` começa declarando o alfabeto. Depois vem o autômato: tipo, nome, estados, inicial, finais e transições, cada uma com origem, destino e símbolo. No fim, o comando `simular` testa as cadeias: `ab` é aceita e `bbb` é rejeitada.
22 — Gramática (EBNF)	**0:17** até 7:13	A linguagem foi formalizada em EBNF. Ela é livre de contexto, tem 11 palavras reservadas, e nelas maiúsculas e minúsculas não fazem diferença.
23 — Decisões de projeto	**0:24** até 7:37	O alfabeto é declarado uma só vez, o que simplifica a verificação. O tipo, AFD ou AFN, é explícito, para checar determinismo só no AFD. A sintaxe é em português, para manter o foco nos conceitos, e há suporte a transições épsilon.
7. Implementação		**2:36**
24 — Roteiro	**0:03** até 7:40	Agora, a implementação.
25 — Pipeline de compilação	**0:22** até 8:02	O arquivo passa pelo analisador léxico, feito com Logos, pelo sintático, com Chumsky, e pelo semântico; depois, segue para o simulador e para a interface, feita com Iced. Se uma fase encontrar erro, a execução para ali e os diagnósticos são exibidos com o Ariadne.
26 — Análise léxica e sintática	**0:24** até 8:26	No léxico, o Logos gera um AFD a partir de anotações: usamos um autômato para ler a linguagem de autômatos. Um caractere inválido é reportado com linha e coluna. No sintático, o Chumsky usa *parser combinators*: a gramática é escrita compondo funções, e o resultado é a AST.
27 — Análise semântica	**0:28** até 8:54	A semântica tem três fases: a tabela de símbolos detecta duplicatas; a verificação de referências encontra estados não declarados, símbolos fora do alfabeto ou autômatos inexistentes; e, só para AFD, verifica-se o determinismo: sem épsilon e com uma transição por estado e símbolo. Os erros são acumulados, então o aluno vê todos de uma vez.
28 — Simulador	**0:35** até 9:29	O simulador guarda todas as configurações intermediárias. Em vez de mostrar a nuvem de estados simultâneos do AFN, ele mostra um caminho único: se a cadeia é aceita, o caminho que aceita; se é rejeitada, o que chegou mais perto da aceitação. O caminho é escolhido por uma busca em largura, e o histórico permite avançar e voltar na simulação.
29 — Interface gráfica	**0:22** até 9:51	A interface desenha o diagrama a partir do texto. A simulação pode ser passo a passo ou automática, destacando o estado atual, a transição usada e o que já foi lido. Dá para arrastar e dar zoom, e o painel fica verde na aceitação e vermelho na rejeição.

Slide	Tempo	Fala
30 — Diagnóstico de erros	**0:22** até 10:13	Cada erro traz linha e coluna, o trecho do código com realce e uma seta no ponto exato, com a descrição em português, como “token inválido” ou “AFD não pode ter epsilon”. Isso troca o tentar e errar visual por um retorno imediato e preciso.
8. Resultados		**2:08**
31 — Roteiro	**0:03** até 10:16	Vamos aos resultados.
32 — Interface: visão geral	**0:17** até 10:33	Esta é a interface com o autômato do exemplo. No topo, escolhemos a simulação; no centro, está o diagrama gerado do arquivo; embaixo, a entrada e os botões de controle.
33 — Simulação: aceitação	**0:22** até 10:55	Executando `ab`: o autômato sai de q_0, lê `a` e vai para q_1, lê `b` e chega em q_2, que é final. A cadeia é aceita e o painel fica verde.
34 — Simulação: rejeição	**0:17** até 11:12	Já com `bbb`, ele fica em q_0, lendo `b` três vezes. A cadeia termina fora de um estado final, então é rejeitada e o painel fica vermelho.
35 — AFN-ε e diagramas grandes	**0:15** até 11:27	A ferramenta também trata AFNs com transições épsilon, à esquerda, e autômatos maiores, como este AFD de 14 estados, navegado com arrastar e zoom.
36 — Testes automatizados	**0:17** até 11:44	Para validar, escrevemos 51 testes automatizados: 12 do léxico, 14 do sintático, 16 do semântico e 9 do simulador. Todos passaram.
37 — Autômatos clássicos	**0:17** até 12:01	Também simulamos autômatos clássicos do Hopcroft e do Sipser, com cadeias aceitas e rejeitadas, incluindo a cadeia vazia. Todos os vereditos bateram com o esperado.
38 — Cenários de erro	**0:20** até 12:21	E criamos arquivos com erros propositais, como um aluno faria. Em todos os casos, a mensagem saiu com linha, coluna, trecho realçado e descrição em português.
9. Conclusão		**1:09**
39 — Roteiro	**0:03** até 12:24	Para concluir.
40 — Contribuições	**0:27** até 12:51	A Autosim aproxima a descrição formal da visualização. O trabalho entrega uma DSL em português, formalizada em EBNF; um compilador pedagógico com diagnósticos localizados; um simulador com navegação nos dois sentidos; e uma interface que anima a simulação. Tudo validado com 51 testes e simulações clássicas, todos corretos.
41 — Limitações e trabalhos futuros	**0:31** até 13:22	Como limitações: um alfabeto por arquivo, um arquivo por execução, sem modo de linha de comando, sem editor embutido e sem conversão de AFN para AFD. Como trabalhos futuros, queremos tratar esses pontos, estender a ferramenta a autômatos com pilha e máquinas de Turing e fazer estudos com estudantes para medir o impacto no ensino.
42 — Obrigado	**0:08** até 13:30	Obrigado pela atenção. Ficamos à disposição para as perguntas da banca.